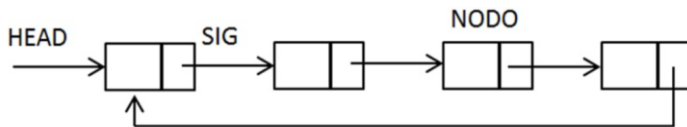


Listas Circulares

La **lista circular** es una especie de lista enlazada simple o doblemente enlazada, pero que posee una característica adicional para el desplazamiento dentro de la lista: **esta no tiene fin**.

Para que la lista no tenga fin, el puntero **siguiente** del último elemento apuntará hacia el **primer elemento** de la lista en lugar de apuntar al valor **NULL**, como hemos visto en el caso de listas enlazadas simples o doblemente enlazadas.

En las listas circulares, **nunca** se llega a una posición en la que ya no sea posible desplazarse. Cuando se llegue al último elemento, el desplazamiento volverá a comenzar desde el primer elemento:



Se pueden entonces definir algunos métodos simples de actualización para una lista circularmente ligada:

agregar(v): inserta un nuevo nodo v inmediatamente después del cursor; si la lista está vacía, entonces v se convierte en el cursor y su apuntador sig. apunta a el mismo.

removeer (): borra y regresa el nodo v inmediatamente después del cursor (no el propio cursor, a menos que este sea el único nodo); si la lista queda vacía, el cursor es puesto a null.

avanzar (): avanza el cursor al siguiente nodo en la lista.

Cómo construir el modelo de un elemento de la lista

Para definir un elemento de la lista, el tipo **struct** será utilizado. El elemento de la lista va a tener un campo **dato** y un puntero **siguiente**. Este debe ser del mismo tipo que el elemento, en caso contrario no podrá apuntar hacia el elemento. El puntero **siguiente** permitirá el acceso al próximo elemento:

```
typedef struct ElementoLista {
    char *dato;
    struct ElementoLista *siguiente;
}Elemento;
```

Para tener el control de la lista es mejor **guardar** ciertos elementos: el primer elemento, el último elemento y el número de elementos. Para ello, otra estructura será utilizada (no es obligatorio, pueden ser utilizadas variables):

```
typedef struct ListaIdentificar {  
    Elemento *inicio;  
    Elemento *fin;  
    int tamaño;  
}Lista;
```

El puntero **inicio** albergará la dirección del primer elemento de la lista.

El puntero **fin** contendrá la dirección del último elemento de la lista.

La variable **tamaño** contiene el número de elementos.

Cualquiera que sea la posición en la lista, los punteros **inicio** y **fin** siempre apuntarán hacia el primer y el último elemento respectivamente.

El campo **tamaño** contendrá el número de elementos de la lista cualquiera sea la operación efectuada sobre la lista.

Cómo se recorre una lista circular y cómo se evitan los ciclos

Mostrar la lista

Para mostrar la lista completa, es necesario posicionarse al inicio de la lista (el puntero **inicio** lo permitirá). Después, utilizando el puntero **siguiente** de cada elemento, la lista es recorrida del primer al último elemento.

En comparación con las listas simples y doblemente enlazadas, en las que la condición para detenerse está dada por el puntero **siguiente** del último elemento, que vale **NULL**, para la lista circular no hay punto de detención a menos que elijamos uno.

A continuación, dos **variantes de visualización**: mostrar la lista (del primer al último elemento) y mostrar la lista sin una condición para detenerse.

Mostrar la lista (del primer al último elemento)

Utilizaremos el tamaño de la lista como la **condición para detenerse**.

```
* mostrar la lista */  
void mostrar (Lista * lista){  
    Elemento *actual;  
    actual = lista->inicio;  
    int i;  
    for(i=0;i<lista->tamaño;++i){
```

```

        printf ("%p - %s\n", actual, actual->dato);
        actual = actual->siguiente;
    }
}

```

Mostrar la lista sin una condición para detenerse (indefinidamente)

La función:

```

/* recorrer la lista indefinidamente*/
void mostrar_indefinidamente (Lista * lista){
    Elemento *actual;
    actual = lista->inicio;
    while (1){
        printf ("%p - %s\n", actual, actual->dato);
        actual = actual->siguiente;
    }
}

```

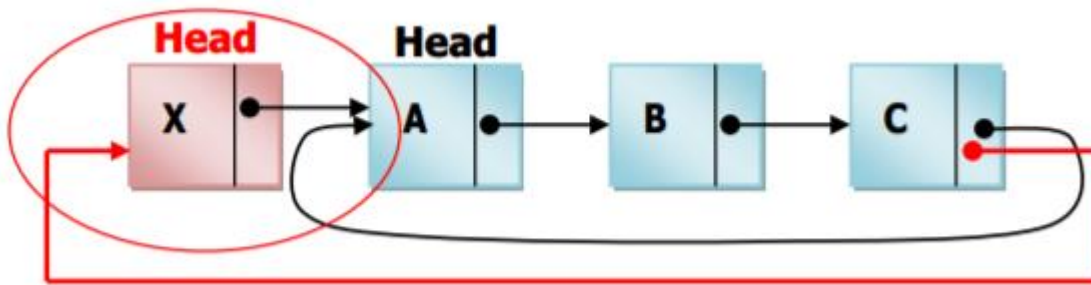
Cómo se identifica a la "cabeza" de la lista circular

Las listas simplemente enlazadas circulares son un poco más simples que las secuenciales ya que la memoria se reserva dinámicamente. En su diseño se cambiaría solo el último nodo de la lista, que ahora no apuntaría a null, simplemente el campo próximo del último elemento hace referencia a la **cabeza** de la lista.



Insertar un elemento al inicio de la lista

Para resolver este problema es necesario determinar si la lista contiene elementos. Si la lista no está vacía se crea el nuevo nodo y se liga con el primer nodo de la lista y head se mueve al nuevo nodo. Es necesario recorrer toda la lista para hacer que el último nodo de la lista apunte al nuevo nodo del inicio. Si la lista está vacía, se crea el primer nodo de la lista ubicando a head en el nuevo nodo.



Pseudocódigo:

Nuevo (p)

Leer (p(dato))

Si head = null entonces

 p(liga) -> p

De lo contrario

 q -> head

 Mientras q(liga) <> head

 q -> q(liga)

 p(liga) -> head

 q(liga) -> p

head -> p

código:

```
/* insertar al inicio de la lista*/
```

```
void insertar_inicio()
```

```
{
```

```
    nodo p, q;
```

```
    p = nuevo();
```

```
    cout << "p(dato) = ";
```

```
    cin >> p->dato;
```

```
    if (head == NULL)
```

```
    {
```

```
        p->liga = p;
```

```
    }
```

```
    else
```

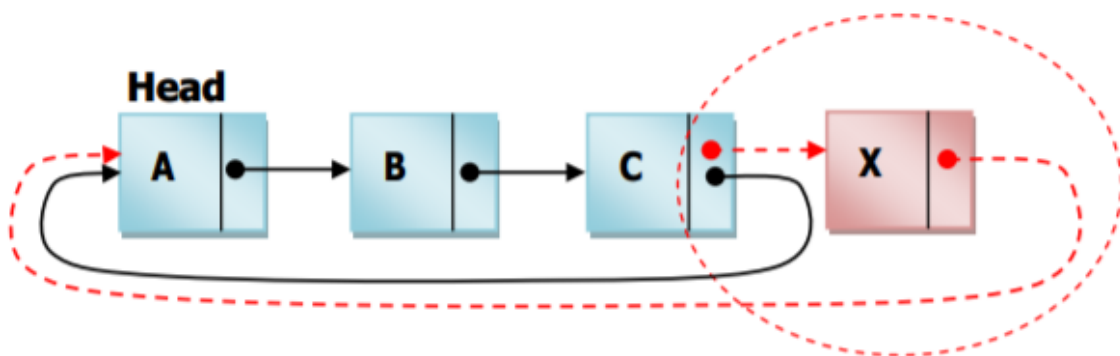
```

{
    q = head;
    while (q->liga != head)
    {
        q = q->liga;
    }
    p->liga = head;
    q->liga = p;
}
head = p;
}

```

Insertar un elemento al final de la lista

Insertar el nuevo nodo al final de la lista y hacer que la liga del nuevo nodo apunte hacia el primer nodo de la lista donde se encuentra head.



```

/* insertar al inicio de la lista*/
void lista_sencilla_circular::insertar_final()
{
    nodo p, q;
    p = nuevo();
    cout << "p(dato) = ";
    cin >> p->dato;
    p->liga = head;
    if (head != NULL)
    {
        q = head;
        while (q->liga != head)

```

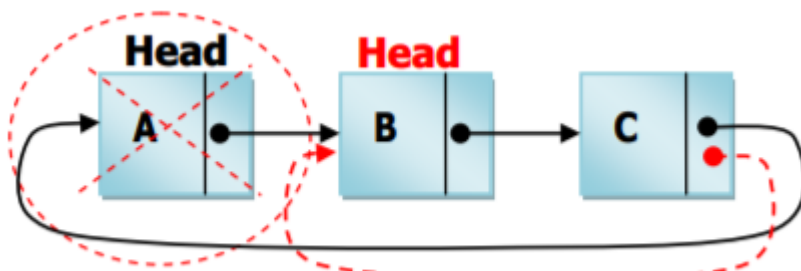
```

        q = q->Liga;
        q->Liga = p;
    }
    else
        head = p;
}

```

Eliminar un elemento al inicio de la lista

Para resolver este problema es necesario determinar si la lista contiene al menos un elemento. Si la lista contiene elementos se posiciona un apuntador en el último nodo de la lista, se elimina el primer nodo de la lista, la variable head se mueve al siguiente nodo de la lista y el último nodo de la lista se liga con el primer nodo de la lista. En caso de que la lista contenga únicamente un nodo, se elimina el nodo y se inicializa la variable head en nulo.



Modelo de la función:

```
int sup_list_circ(Lista *lista);
```

La función devuelve -1 en caso de un error, si no devuelve 0.

Etapas: el puntero sup_elemento contendrá la dirección del elemento 1, el puntero inicio apuntará hacia el segundo elemento, el puntero siguiente del último elemento apuntará hacia el primer elemento (que era el segundo antes de la eliminación), el tamaño de la lista disminuirá 1 elemento:

La función:

```

/* eliminación al inicio de la lista */
int sup_list_circ(Lista * lista){
    if (lista->tamaño < 2)

```

```

    return -1;
Elemento *sup_element;

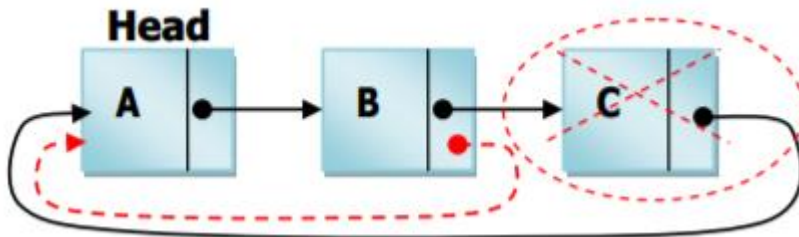
sup_elemento = lista->inicio;
lista->inicio = lista->inicio->siguiente;
lista->fin->siguiente = lista->inicio;

free (sup_elemento->dato);
free (sup_elemento);
lista->tamaño--;
return 0;
}

```

Eliminar un elemento al final de la lista

Para resolver este problema es necesario determinar si la lista contiene al menos un elemento. Es necesario recorrer los nodos de la lista para ubicarse en la penúltima posición, eliminar el último nodo y hacer que la liga del penúltimo nodo apunte hacia el primer nodo de la lista. En el caso de que la lista contenga únicamente un nodo, se elimina el nodo y se inicializa la variable Head en nulo.



Solución en pseudocódigo

```
Si head <> null entonces
  p ← head
  Mientras (p(liga) <> head)
    [ q ← p
      p ← p(liga)
    Si p<>head entonces
      [ q(liga) ← head
    De lo contrario
      [ head ← null
  Eliminar (p)
De lo contrario
  [ Mensaje ('No hay elementos...')
```

```
/* eliminación al Final de la Lista */
void lista_sencilla_circular::borrar_ultimo()
{
    nodo p, q;
    if (head != NULL)
    {
        p = head;
        while (p->liga != head)
        {
            q = p;
            p = p->liga;
        }
        if (p != head)
            q->liga = head;
        else
            head = NULL;
        delete (p);
    }
    else
        cout << "Lista Vacía";
}
```


Bibliografía de las fuentes consultadas

http://uabcs.mx/difusion2017/files/libros/pdf/193_20160910055537.pdf

<https://sites.google.com/site/edatosubo/4-estructura-de-datos-dinamicas/1-listas/3-listas-circulares>